Section

1. Introduction
2. Objectives and Requirements
3. Tools and Environment
4. Wireframe Design (Figma)
5. Source Code
6. Code Explanation (6.1 to 6.7)
7. Running the Application
8. Requirements Mapping to Marking Criteria
9. GitHub Submission
10. Conclusion

References

1. Introduction

This document describes the design, implementation and testing of a basic interactive graphical user interface (GUI) application built with Java Swing. The application, named SwingAssignment, displays a welcome message inside a window and lets the user interact with it through two buttons: one changes the text of the label and the other changes the background colour of the panel.

Swing is Java's lightweight GUI toolkit and follows an event-driven model: the program builds its components, then waits for user actions (such as button clicks) which are handled by event listeners. The assignment demonstrates the core Swing building blocks (JFrame, JPanel and JLabel), the use of inheritance and a constructor, the use of ImageIcon for a custom application icon, and basic event handling.

2. Objectives and Requirements

The assignment brief requires the following, and the table in Section 8 maps each to the implementation.

- Create a basic Java Swing GUI that uses a JFrame, JLabel and JPanel.
- Implement inheritance and a constructor.
- Create a class called SwingAssignment with JFrame variable frame, JPanel variable panel and JLabel variable label.
- Set the label text to "Welcome to Java Swing First Individual Assignment by [Your name]", add the label to the panel and the panel to the frame.
- Set the frame title to "Java Swing Assignment", the size to 800x800 pixels, and close the application when the close button is clicked.
- Use an own application icon through ImageIcon.
- Addition: add functionality to change the JLabel text and the JPanel background colour.
- Create a wireframe in Figma and push all files to GitHub.

3. Tools and Environment

Tool	Purpose
Figma	Designing the wireframe of the interface
IntelliJ IDEA / VS Code	Writing, compiling and running the Java code
Java Development Kit (JDK 8 or later)	Compiling and running the Swing application
Git and GitHub	Version control and submission (lecturer added as collaborator)

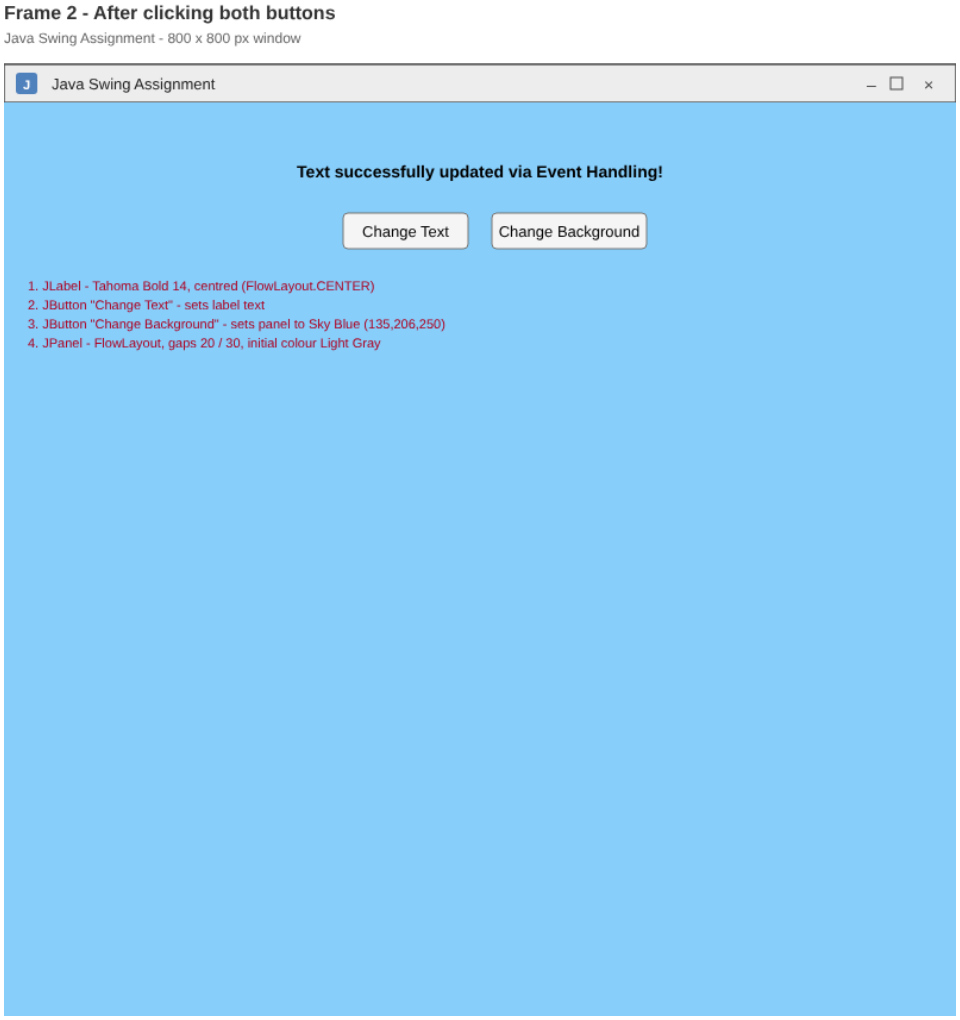
4. Wireframe Design (Figma)

Before coding, a wireframe of the window was created in Figma to plan the layout. The design is a single 800x800 frame containing one panel. The welcome label sits at the top centre of the panel, with the two buttons ("Change Text" and "Change Background") placed beside or below it, matching the FlowLayout used in the code.

Wireframe Element	Description
Frame (800 x 800)	Window titled "Java Swing Assignment" with the custom icon in the title bar
Panel	Full-size container with a light grey background (initial state)
Label	Bold Tahoma 14 text: "Welcome to Java Swing First Individual Assignment by Ranny Brima"
Button 1	"Change Text" – updates the label text
Button 2	"Change Background" – changes the panel to sky blue

[Insert screenshot: Figma wireframe – initial state]

[Insert screenshot: Figma wireframe – after button clicks (optional second state)]



5. Source Code

File name: SwingAssignment.java. The file icon.png must be placed in the project root folder so the application icon loads.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SwingAssignment extends JFrame {

    // Declare the required variables
    private JFrame frame;
    private JPanel panel;
    private JLabel label;
    private JButton changeTextButton;
    private JButton changeColorButton;

    // Constructor implementing inheritance and setup
    public SwingAssignment() {
        // Instantiate JFrame variable named 'frame' using 'this' (fulfilling inheritance)
        frame = this;
        frame.setTitle("Java Swing Assignment");
        frame.setSize(800, 800);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null); // Centers the window on screen

        // 1. ImageIcon addition (Make sure an 'icon.png' file is in your project root folder)
        ImageIcon appIcon = new ImageIcon("icon.png");
        frame.setIconImage(appIcon.getImage());

        // Instantiate JPanel variable named 'panel'
        panel = new JPanel();
        panel.setLayout(new FlowLayout(FlowLayout.CENTER, 20, 30));
        panel.setBackground(Color.LIGHT_GRAY);

        // Instantiate JLabel variable named 'label' with the required text
        label = new JLabel("Welcome to Java Swing First Individual Assignment by Ranny Brima");
        label.setFont(new Font("Tahoma", Font.BOLD, 14));

        // Instantiate interactive buttons (Fulfilling the addition requirement)
        changeTextButton = new JButton("Change Text");
        changeColorButton = new JButton("Change Background");

        // Add functionality to change the text of the JLabel
        changeTextButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                label.setText("Text successfully updated via Event Handling!");
            }
        });

        // Add functionality to change the background of the JPanel
        changeColorButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                panel.setBackground(new Color(135, 206, 250)); // Sky Blue
            }
        });

        // Add the label and buttons to the panel
        panel.add(label);
        panel.add(changeTextButton);
```

```
        panel.add(changeColorButton);

        // Add the panel to the frame
        frame.add(panel);

        // Make the frame visible
        frame.setVisible(true);
    }

    // Main method to run the application on the Event Dispatch Thread
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new SwingAssignment();
            }
        });
    }
}
```

6. Code Explanation

6.1 Imports

javax.swing.* provides JFrame, JPanel, JLabel, JButton, ImageIcon and SwingUtilities. java.awt.* provides Color, Font and FlowLayout. java.awt.event.ActionEvent and ActionListener provide the event-handling types.

6.2 Class Declaration and Inheritance

The class is declared as public class SwingAssignment extends JFrame. This is inheritance: SwingAssignment is a subclass of JFrame and automatically gains all of its methods, such as setTitle(), setSize() and setVisible(). The required variable named frame is instantiated by assigning this (the current JFrame object) to it, so the class itself is the window.

6.3 Variables

The class declares frame (JFrame), panel (JPanel), label (JLabel), and two JButton variables, changeTextButton and changeColorButton. They are private fields so they are accessible throughout the class, including inside the event listeners.

6.4 Constructor

The public SwingAssignment() constructor runs automatically when an object is created and builds the whole interface. It performs these steps in order:

- Sets the window title to "Java Swing Assignment" and the size to 800x800 pixels.
- Sets JFrame.EXIT_ON_CLOSE so the program terminates when the close button is clicked.
- Calls setLocationRelativeTo(null) to centre the window on the screen.
- Creates an ImageIcon from icon.png and passes its image to setIconImage() to use the custom application icon.
- Creates the panel with a FlowLayout (centre aligned, 20 px horizontal and 30 px vertical gaps) and a light grey background.
- Creates the label with the required welcome text and sets its font to Tahoma, bold, size 14.

- Creates the two buttons, attaches their listeners, adds the label and buttons to the panel, adds the panel to the frame, and finally makes the frame visible.

6.5 ImageIcon

ImageIcon loads an image file so it can be used in Swing components. Here new ImageIcon("icon.png") loads the student's own icon and getImage() converts it to an Image, which setIconImage() displays in the window's title bar and taskbar.

6.6 Event Handling (Addition)

Each button has an ActionListener registered with addActionListener(). When a button is clicked, Swing calls the listener's actionPerformed() method:

- Change Text: calls label.setText(...) to replace the welcome message with "Text successfully updated via Event Handling!".
- Change Background: calls panel.setBackground(new Color(135, 206, 250)) to change the panel from light grey to sky blue.

6.7 The main Method

main() starts the program using SwingUtilities.invokeLater(), which creates the SwingAssignment object on the Event Dispatch Thread (EDT). Swing components are not thread-safe, so all GUI creation and updates should happen on the EDT.

7. Running the Application

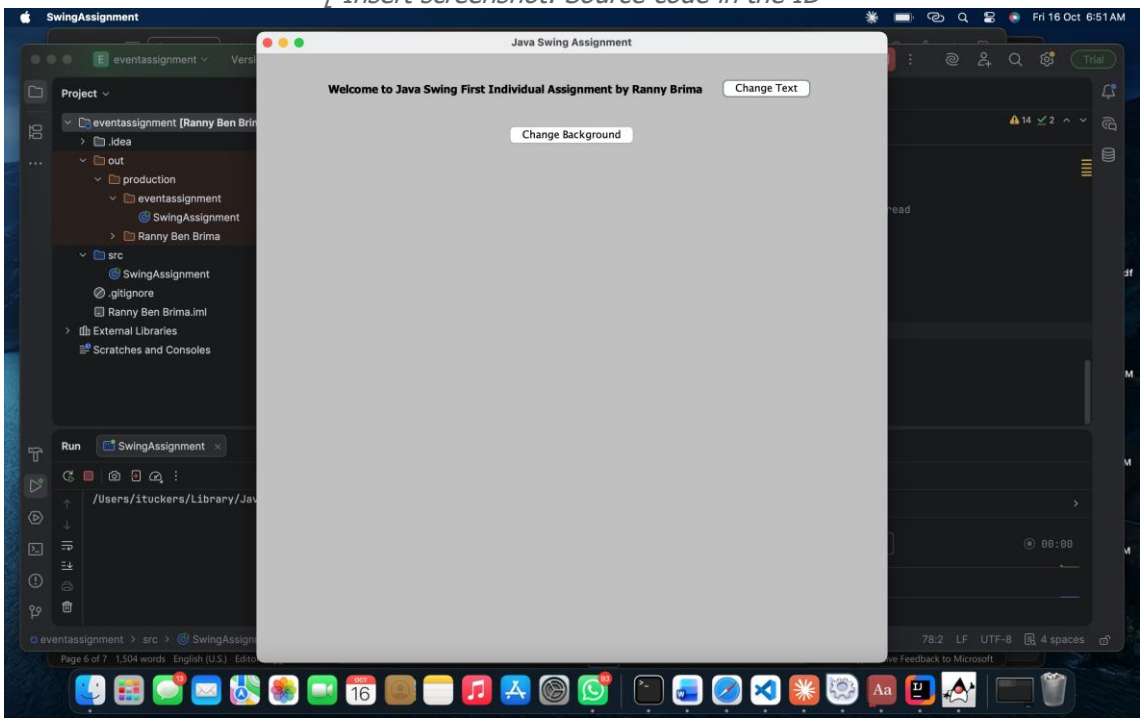
- Create a project in IntelliJ IDEA or VS Code and add SwingAssignment.java.
- Place icon.png in the project root folder.
- Run the main method (Run ▶ in the IDE, or javac SwingAssignment.java then java SwingAssignment in a terminal).
- Click "Change Text" and "Change Background" to test the added functionality.

[Insert screenshot: Application on start-up (grey panel, welcome label, custom icon)]

[Insert screenshot: After clicking "Change Text"]

[Insert screenshot: After clicking "Change Background"]

[Insert screenshot: Source code in the ID



8. Requirements Mapping to Marking Criteria

Criteria	Where it is implemented	Marks
JFrame	Class extends JFrame; frame variable; title, 800x800 size, EXIT_ON_CLOSE	20
JPanel	panel variable instantiated, background colour, label and buttons added, panel added to frame	20
JLabel	label variable with the required welcome text, font set, added to panel	20
ImageIcon	ImageIcon("icon.png") used with setIconImage()	10
Wireframe	Figma wireframe (Section 4)	20
Documentation and GitHub	This document; all files committed and pushed to GitHub	10
Total		100

9. GitHub Submission

The project was version-controlled with Git and pushed to GitHub. The typical steps were:

- `git init`
- `git add .`
- `git commit -m "Add SwingAssignment Java Swing GUI"`
- `git branch -M main`
- `git remote add origin <repository-url>`

- `git push -u origin main`

The lecturer (Amandus Benjamin Coker) was then added as a collaborator under Settings > Collaborators in the repository. Repository link: [Insert your repository link].

[Insert screenshot: GitHub repository showing all committed files and the lecturer as collaborator]

10. Conclusion

The SwingAssignment application meets the brief: it uses a JFrame, JPanel and JLabel, applies inheritance by extending JFrame, uses a constructor to build the interface, loads a custom icon with ImageIcon, and adds interactivity through event listeners that change the label text and panel background. The assignment demonstrated the event-driven programming model, in which the interface reacts to user actions.

References

Oracle (n.d.) The Java Tutorials: Creating a GUI with Swing. Available at: https://docs.oracle.com/javase/tutorial/uiswing/ (Accessed: October 2026).